Chapitre 7

Détection de fraude

Plan du Chapitre

27 Le problème de la fraude

28 Le déséquilibre extrême

29 Algorithmes de détection d'anomalies non supervisée

30 Métriques d'évaluation

Objectifs pédagogiques

À la fin de ce chapitre, l'étudiant doit être capable de :

1. Expliquer pourquoi la fraude est un problème de classification particulier.
2. Choisir et appliquer correctement une stratégie contre le déséquilibre, sans fuite de données.
3. Évaluer un détecteur avec des métriques adaptées aux événements rares et à un budget d'analyse.
4. Expliquer le fondement théorique de SMOTE, d'Isolation Forest, de LOF, de DBSCAN, du One-Class SVM et de l'autoencodeur.
5. Valider un modèle de façon chronologique et fixer un seuil selon un coût exprimé en montants.

Les formes de fraude

Domaine	Exemples	Données typiques
Paiement par carte	Carte volée, carte clonée, paiement à distance frauduleux	Transactions, commerçant, pays, canal
Paiement mobile et virement	Prise de contrôle de compte, arnaque au faux conseiller	Transferts, appareil, historique de connexion
Assurance	Sinistre fictif ou gonflé, accident arrangé	Déclarations, expertises, historique de sinistres
Crédit	Usurpation d'identité, faux justificatifs	Dossiers de demande, données déclaratives
Blanchiment	Fractionnement de dépôts, circuits de transferts	Flux entre comptes, réseaux de relations

Six caractéristiques qui changent tout

1. **Déséquilibre extrême :** Souvent moins de 1% de fraudes, parfois moins de 0,1%.
2. **Coûts asymétriques et variables :** Une fraude manquée coûte son montant ; une fausse alerte coûte un temps d'analyse et un client mécontent.
3. **Adversaire actif :** Les fraudeurs s'adaptent aux règles de détection : les schémas changent dans le temps (concept drift).
4. **Étiquettes tardives et incomplètes :** Une fraude est souvent confirmée des semaines plus tard (contestation du porteur). Les fraudes jamais détectées restent étiquetées « normales ».
5. **Contrainte de temps réel :** Une transaction par carte doit être autorisée ou refusée en quelques dizaines de millisecondes.
6. **Capacité d'analyse limitée :** Une équipe ne peut examiner qu'un nombre fixe d'alertes par jour.

Les grandes approches

Approche	Principe	Atouts
Règles métier	« Montant $>$ 1 000 € et pays inhabituel $\rightarrow$ alerte »	Lisibles, immédiates
Supervisée	Apprendre à partir des fraudes passées étiquetées	Très performante sur les schémas connus
Non supervisée	Repérer ce qui s'écarte du comportement normal	Détecte des schémas inédits, pas besoin d'étiquettes
Hybride	Règles + score supervisé + score d'anomalie	Combine les forces
Graphes	Analyser les liens entre comptes, appareils, commerçants	Indispensable pour les réseaux organisés et le blanchiment

Pourquoi c'est un problème

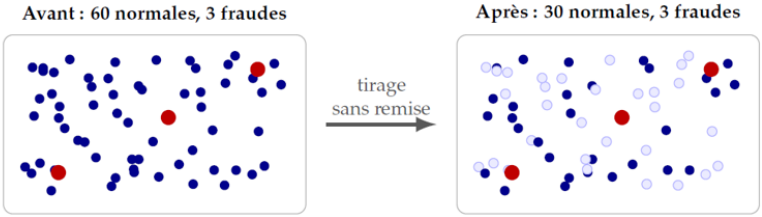
Avec 0,17 % de fraudes, un modèle qui ne détecte rien atteint 99,83 % d'accuracy. Pendant l'apprentissage, la perte moyenne est dominée par la classe majoritaire.

Les leviers disponibles

Niveau	Technique	Principe
Données	Sous-échantillonnage aléatoire	Retirer une partie des transactions normales.
Données	Sur-échantillonnage aléatoire	Dupliquer des exemples de fraude.
Données	SMOTE, Borderline-SMOTE, ADASYN	Créer des exemples de fraude synthétiques par interpolation.
Données	Nettoyage	Retirer les exemples ambigus près de la frontière de décision.
Algorithme	Pondération des classes	Donner plus de poids aux erreurs sur la classe rare.
Décision	Ajustement du seuil	Garder le modèle et déplacer le seuil d'alerte.

Le sous-échantillonnage aléatoire

Principe : on garde toutes les fraudes et on tire au hasard une partie seulement des transactions normales, jusqu'à atteindre un ratio cible (par exemple 1 fraude pour 10 normales).



Points pâles : transactions normales écartées (non utilisées pour l'entraînement).

Exemple : 100 000 transactions dont 200 fraudes (0,2 %). Avec un ratio 1 :10, on garde les 200 fraudes et 2 000 normales tirées parmi 99 800 : le modèle apprend sur 2 200 lignes dont 9,1% de fraudes.

Algorithme :

Entrées: jeu d'**entraînement** $D = \{(x_i, y_i)\}$, ratio cible r (normales par fraude)
Sorties: jeu rééquilibré D' et taux de conservation β

1 $F \leftarrow \{i : y_i = 1\}$; // toutes les fraudes
2 $N \leftarrow \{i : y_i = 0\}$; // toutes les normales
3 $m \leftarrow \min(|N|,\ r \times |F|)$; // nombre de normales à garder
4 $N' \leftarrow$ tirage aléatoire **sans remise** de m indices dans N;
5 $D' \leftarrow$ mélange aléatoire de $F \cup N'$;
6 $\beta \leftarrow m/|N|$; // part des normales conservées
7 **retourner** D', β;

1. Sous-échantillonner après la séparation entraînement / validation / test.
2. Ne toucher qu'au jeu d'entraînement : la validation et le test gardent la proportion réelle.
3. En validation croisée, refaire le tirage dans chaque pli (pipeline imblearn).

Avantages

- Entraînement beaucoup plus rapide (2 200 lignes au lieu de 100 000).
- Aucune donnée artificielle, contrairement à SMOTE.
- Simple à expliquer et à auditer.

Inconvénients

- Perte d'information sur les comportements normaux.
- Le résultat dépend du tirage : répéter avec plusieurs graines.
- Les probabilités sont faussées et doivent être corrigées.

Choix du ratio

Tester 1 :1, 1 :5, 1 :10 et 1 :20 sur la validation (PR-AUC), puis comparer au modèle sans rééchantillonnage. Le ratio 1 :1 est rarement le meilleur.

```
from imblearn.pipeline import Pipeline
from imblearn.under_sampling import RandomUnderSampler
pipe = Pipeline([
    ("sous_ech", RandomUnderSampler(sampling_strategy=0.1, random_state=42)),  # 1:10
    ("modele",   LogisticRegression(max_iter=1000))])
pipe.fit(X_train, y_train)    # le tirage ne s'applique qu'à l'entraînement
```

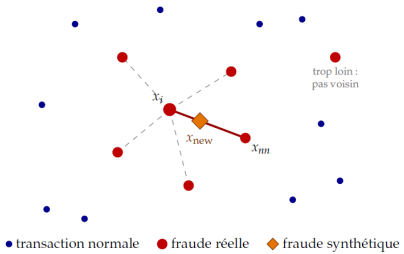
SMOTE : fondement et algorithme

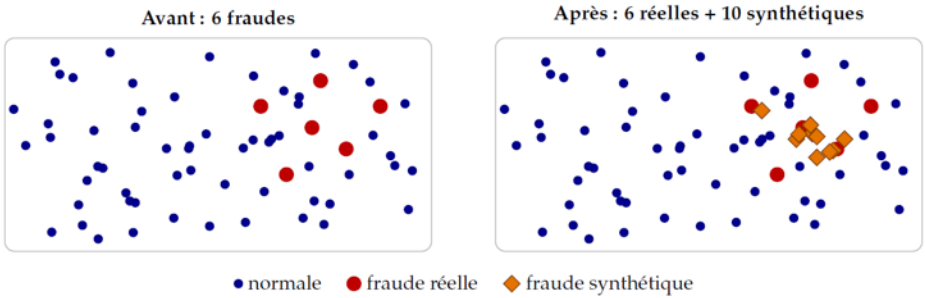
Principe : Dupliquer quelques fraudes peut faire surapprendre le modèle. SMOTE (*Synthetic Minority Over-sampling Technique*) génère des fraudes plausibles entre des exemples existants, en supposant que la zone entre deux fraudes voisines appartient aussi à la classe fraude.

Algorithme : Pour créer G fraudes synthétiques :

1. Pour chaque fraude x_i, trouver ses k plus proches voisins parmi les fraudes (souvent $k = 5$).
2. Répéter G fois :
   1. Tirer une fraude x_i et l'un de ses voisins x_{nn}.
   2. Tirer λ uniformément dans $[0, 1]$.
   3. Créer $x_{\text{new}} = x_i + \lambda(x_{\text{nn}} - x_i)$.

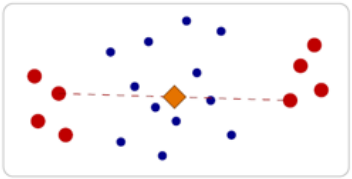
Le nouveau point se trouve sur le segment entre x_i et x_{nn}.





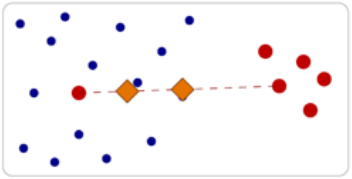
Les nouveaux points remplissent la zone des fraudes au lieu de répéter les mêmes points : le modèle apprend une région de fraude plus large et plus régulière, avec moins de surapprentissage qu'avec une simple duplication.

1. Fraudes en plusieurs groupes



Avec $k = 5$ et seulement 4 fraudes par groupe, un voisin est forcément pris dans l'autre groupe : le point synthétique tombe **en pleine zone normale**.

2. Une fraude isolée (bruit)



Une fraude atypique noyée parmi les normales sert quand même de point de départ : SMOTE **propage le bruit** dans la zone normale.

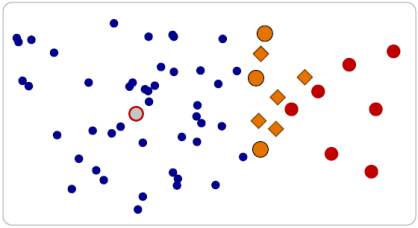
Dans les deux cas, SMOTE **ignore les transactions normales** : il ne regarde que les fraudes. Les variantes suivantes corrigent ce défaut.

Les variantes de SMOTE

Borderline-SMOTE : La méthode cible les fraudes proches de la frontière de décision. Pour chaque fraude, on examine ses m plus proches voisins et on compte m', le nombre de voisins normaux.

m'	**Statut**	**Génération**
$m' = m$	Bruit : entourée de normales	Non
$m/2 \leq m' < m$	En danger : près de la frontière	Oui
$m' < m/2$	Sûre : au cœur des fraudes	Non

Exemple si $m = 5$: bruit (5/5) et sûre (0–2/5) ignorées ; en danger (3–4/5) utilisée pour générer des points synthétiques.



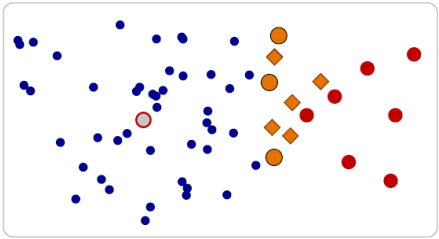
- ○ **Bruit** : 5 voisins normaux sur 5 ⇒ ignorée.
- ● **En danger** : 3 ou 4 sur 5 ⇒ **seules utilisées**.
- ● **Sûre** : 0 à 2 sur 5 ⇒ inutile.

◆ les points synthétiques renforcent la **frontière**, là où le modèle hésite.

ADASYN : pour chaque fraude

1. Mesurer sa difficulté : part de normales parmi ses 5 voisins.
2. Répartir les points à créer en proportion de cette difficulté.
3. Créer les points comme avec SMOTE, sur des segments.

Les fraudes au cœur du groupe reçoivent 0, celles proches de la frontière de 1 à 3.



Piège : la fraude isolée à gauche, sans doute du bruit, en reçoit le plus (4). D'où l'intérêt de nettoyer les données avant ou après.

La règle : rééchantillonner uniquement l'entraînement

Le rééchantillonnage s'applique après la séparation, sur le seul jeu d'entraînement, et à l'intérieur de chaque pli de validation croisée.

Après rééchantillonnage, le modèle croit que la fraude est beaucoup plus fréquente qu'en réalité : ses probabilités sont surestimées.

Si l'on garde une proportion β des transactions normales, la cote de fraude observée pendant l'entraînement est multipliée par $1/\beta$.

Par le théorème de Bayes, la cote réelle s'obtient en multipliant la cote apprise par β :

$$\frac{p}{1-p} = \beta \frac{p_s}{1-p_s} \qquad \Longleftrightarrow \qquad p = \frac{\beta p_s}{\beta p_s - p_s + 1}$$

Que choisir ?

1. Commencer par ne rien rééchantillonner et ajuster le seuil.
2. Essayer ensuite la pondération des classes, simple et sans données artificielles.
3. Le sous-échantillonnage est utile quand le jeu est très volumineux : il accélère fortement l'entraînement.
4. SMOTE et ses variantes n'apportent pas toujours de gain, en particulier avec les modèles d'arbres boostés. Ils se justifient par une amélioration mesurée en validation, pas par principe.

Algorithmes de détection d'anomalies non supervisée

Ces méthodes n'utilisent pas les étiquettes. Elles apprennent ce qu'est un comportement « normal » et attribuent à chaque point un score d'anomalie $a(x)$: plus il est élevé, plus le point est suspect.

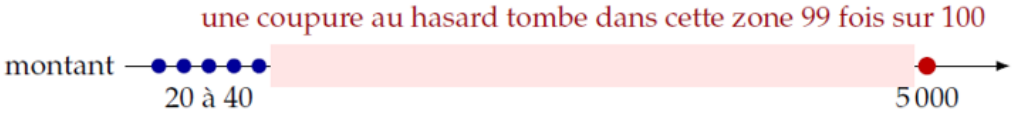
Deux usages :

1. Détection d'anomalies (outlier detection) : les données d'entraînement contiennent déjà quelques anomalies, qu'on cherche à repérer.
2. Détection de nouveauté (novelty detection) : on apprend sur des données jugées normales, puis on score de nouvelles transactions.

1. Isolation Forest

Hypothèse : Les anomalies sont rares et différentes. Elles sont donc plus faciles à isoler que les points normaux : il faut peu de coupures aléatoires pour les séparer du reste.

Exemple : montants de 6 transactions d'un client (en dinars).

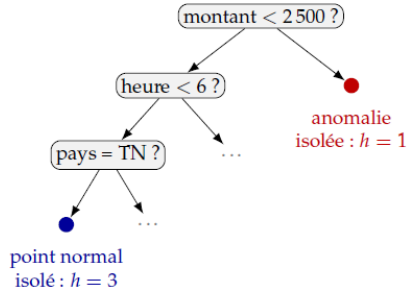


- On coupe au hasard entre le minimum (20) et le maximum (5 000).
- La coupure isole presque toujours 5 000 du premier coup.
- Pour isoler 30, il faut au contraire plusieurs coupures.

==> moins il faut de coupures pour isoler un point, plus il est suspect.

Construction d'un arbre d'isolement (iTree) :

1. Tirer un sous-échantillon de ψ points (par défaut $\psi = 256$).
2. Construire récursivement l'arbre tant que le nœud contient plus d'un point et que la hauteur limite $\ell = \lceil \log_2(\psi) \rceil$ n'est pas atteinte :
   1. choisir une variable q au hasard ;
   2. choisir un seuil p uniformément entre le minimum et le maximum de q dans le nœud ;
   3. envoyer à gauche les points avec $x_q < p$, à droite les autres.
3. Répéter pour construire t arbres (par défaut $t = 100$).



Du nombre de coupures au score d'anomalie :

1. Calculer la profondeur moyenne de x sur les 100 arbres.
2. La comparer à la profondeur habituelle d'un point quelconque : environ 10 coupures pour un échantillon de 256 points.
3. Calculer le rapport r :

$$r = \frac{\text{profondeur moyenne de } x}{\text{profondeur habituelle}}$$

Le score d'anomalie est alors :

$$s(x) = 2^{-r}$$

Profondeur moyenne de x	Rapport	Score $s(x)$	Lecture
2 coupures	0,2	0,87	très suspect
6 coupures	0,6	0,67	à surveiller
10 coupures	1	0,50	rien de particulier
13 coupures	1,3	0,41	point normal

Pourquoi de petits échantillons (256 points) ?

1. Avec trop de points, des fraudes proches les unes des autres se « cachent » mutuellement et deviennent difficiles à isoler.
2. Avec moins de points, elles ressortent mieux, et l'algorithme est très rapide.

Avec scikit-learn :

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(n_estimators=100, max_samples=256,
                      random_state=42).fit(X_train)
score = -iso.score_samples(X_test)   # = s(x) : plus il est grand,
                                     #   plus la transaction est suspecte
```

2. Local Outlier Factor (LOF)

Idée clé : être loin des autres ne suffit pas. Ce qui compte, c'est d'être plus isolé que ses voisins. LOF compare donc chaque point à son voisinage.



- **A** est assez proche du groupe dense, mais ses voisins sont serrés les uns contre les autres : A est à l'écart de son groupe ⇒ suspect.
- **B** est un peu plus loin de ses voisins, mais dans le groupe diffus tous les points sont espacés : B est normal.

Mesurer la densité autour d'un point :

1. Choisir un nombre de voisins k (par exemple $k = 3$).
2. Pour chaque point, trouver ses k plus proches voisins.
3. Calculer la distance moyenne entre le point et ses k voisins.

Distance moyenne aux voisins	Densité locale
petite (voisins très proches)	**élevée** : le point est dans une zone dense
grande (voisins éloignés)	**faible** : le point est dans une zone vide

$$\text{Densité locale} \approx \frac{1}{\text{distance moyenne aux } k \text{ voisins}}$$

LOF utilise une distance légèrement « lissée » pour que deux points presque confondus ne donnent pas une densité infinie.

$$\text{LOF}(x) = \frac{\text{densité moyenne des voisins de } x}{\text{densité de } x}$$

Point	Distance de x	Distance des voisins	LOF	Lecture
A	1,0	0,34	**2,5**	**anomalie**
B	1,2	1,2	**1,0**	même densité : normal

Interprétation :

- LOF ≈ 1 : le point ressemble à ses voisins, donc il est normal.
- LOF > 1 : le point est plus isolé que ses voisins ; plus le LOF est grand, plus il est suspect. En pratique, on examine de près les valeurs supérieures à 1,5.
- LOF < 1 : le point se trouve au cœur d'un groupe dense.

Choix de k :

- k trop petit : le score varie beaucoup d'un point à l'autre (sensible au bruit).
- k trop grand : on compare le point à des points lointains, on perd l'aspect « local ».

Valeur par défaut dans scikit-learn : k = 20.
Points forts : détecte les anomalies locales quand les groupes ont des densités différentes.
Points faibles : lent sur de gros volumes (calcul des voisins) ; sensible à l'échelle des variables ⇒ normaliser avant.

Avec scikit-learn :

```
from sklearn.neighbors import LocalOutlierFactor
lof = LocalOutlierFactor(n_neighbors=20, novelty=True).fit(X_train)
score = -lof.score_samples(X_test)    # = LOF : proche de 1 -> normal,
                                      #   nettement > 1 -> suspect
```

novelty=True permet de noter de nouvelles transactions, absentes de l'entraînement.

3. DBSCAN

Principe : DBSCAN (Density-Based Spatial Clustering of Applications with Noise) est d'abord un algorithme de clustering : il regroupe les points situés dans des zones denses. Les points qui n'appartiennent à aucune zone dense sont étiquetés bruit. En détection de fraude, ce bruit est un ensemble de candidats anomalies.

Deux paramètres :

1. ε : le rayon du voisinage.
2. MinPts : le nombre minimal de points dans ce voisinage pour former une zone dense.

Les points centraux (core points)

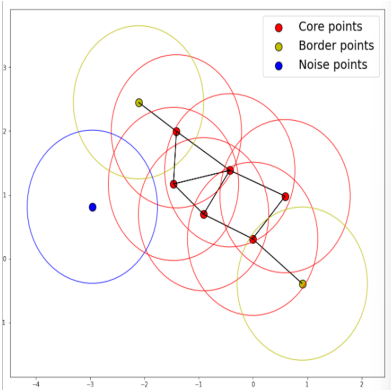
- Ce sont les points qui se trouvent au cœur d'un groupe
- Leur voisinage doit contenir plus que *m* points

Les points frontière (border points)

- Ce sont les points qui bordent un groupe
- Ils sont voisins d'un point central mais ne sont pas centraux eux-mêmes

Les points aberrants (noise points)

- Ce sont les points isolés
- Ils ne sont ni centraux, ni frontière



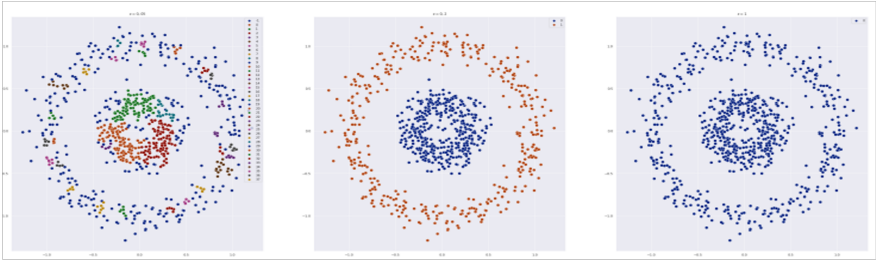
Algorithme :

1. Calculer le ε-voisinage de chaque point et repérer les points cœurs.
2. Pour chaque point cœur non étiqueté, créer un cluster et ajouter progressivement les points atteignables par densité. N'étendre la recherche qu'à partir des points cœurs.
3. Étiqueter comme bruit les points restés sans cluster.

Choisir les paramètres :

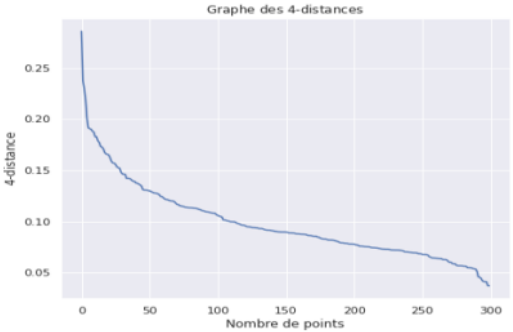
1. En dimension d, choisir MinPts $\geq d + 1$; la règle courante est MinPts $= 2d$.
2. Pour choisir ε, tracer la courbe des distances au k-ième voisin le plus proche, avec $k = \text{MinPts} - 1$, puis retenir la valeur au « coude », là où la distance augmente brusquement. Les points au-delà du coude seront considérés comme du bruit.

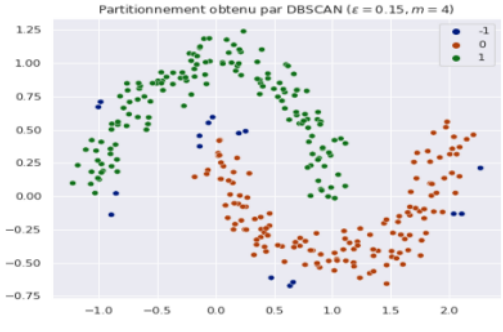
Quel rayon ε du voisinage ?



ε trop faible : nombreux groupes de petite taille
ε trop élevé : un unique groupe

Peut-on régler automatiquement une valeur raisonnable pour ε ?





Avec scikit-learn :

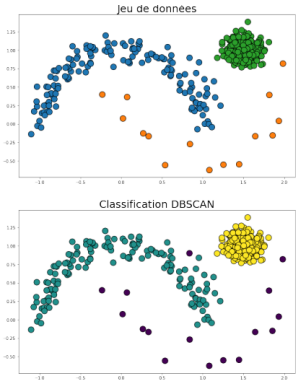
```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=10).fit(X_scaled)  # normaliser avant !
anomalie = (db.labels_ == -1)   # étiquette -1 = bruit = transaction suspecte
```

Points forts :

1. **Repérage des données aberrantes :** DBSCAN étiquette comme bruit les points qui n'appartiennent à aucun cluster. On peut ainsi les examiner ou les exclure avant une analyse ultérieure.
2. **Groupes de formes variées :** les clusters peuvent être non convexes et ne sont pas limités à des groupes linéairement séparables, contrairement aux partitions souvent produites par k-means.
3. **Nombre de groupes non fixé à l'avance :** DBSCAN le déduit des composantes connectées par densité.

Points faibles :

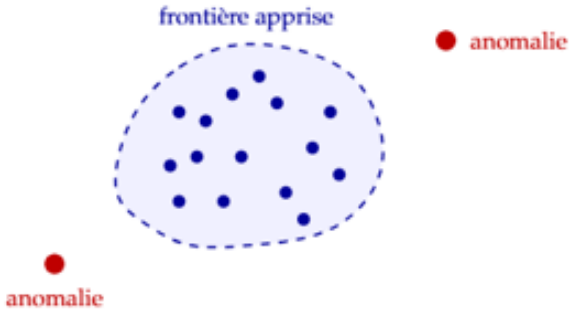
1. **Densités variables :** un seul seuil ε est utilisé pour tous les groupes ; il peut donc être inadapté lorsque leurs densités diffèrent.
2. **Points de faible densité :** ils peuvent être étiquetés comme bruit, même s'ils correspondent à des observations pertinentes. DBSCAN les signale, mais ne les supprime pas automatiquement.
3. **Nouveaux points :** DBSCAN est transductif et ne fournit pas de règle native pour classer une nouvelle observation ; une stratégie complémentaire ou un nouveau partitionnement est nécessaire.



4. One-Class SVM : tracer la frontière du normal

Principe : on ne montre au modèle que des transactions normales. Il apprend une frontière qui les entoure au plus juste. Toute transaction qui tombe en dehors est suspecte.

- À l'intérieur : comportement habituel ⇒ accepté.
- À l'extérieur : comportement jamais vu ⇒ alerte.
- Plus le point est loin de la frontière, plus il est suspect.



Réglage	**Ce qu'il contrôle**	**En pratique**
ν	Part maximale des données d'entraînement que l'on accepte de laisser hors de la frontière.	$\nu = 0{,}01$: environ 1 % des points laissés dehors. À rapprocher du taux d'alerte que l'équipe peut traiter.
Noyau RBF et γ (gamma)	Souplesse et forme de la frontière ; le noyau RBF permet des formes courbes.	γ trop grand : frontière trop collée aux données (surapprentissage) ; trop petit : frontière trop large.

Avec scikit-learn :

```
from sklearn.svm import OneClassSVM
from sklearn.preprocessing import StandardScaler
sc = StandardScaler().fit(X_normales)           # normaliser : indispensable
oc = OneClassSVM(kernel="rbf", nu=0.01, gamma="scale")
oc.fit(sc.transform(X_normales))
score = -oc.decision_function(sc.transform(X_test))  # grand = suspect
alerte = oc.predict(sc.transform(X_test)) == -1      # -1 = hors frontière
```

Points forts :

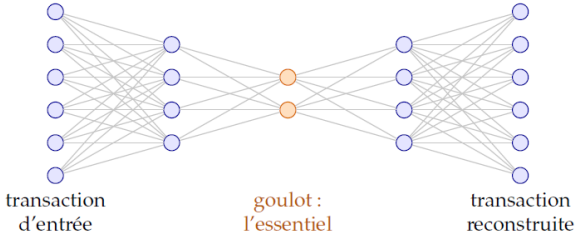
1. Frontières de formes complexes.
2. Efficace sur des volumes moyens et en grande dimension.
3. Sait noter de nouvelles transactions.

Points faibles :

1. Lent au-delà de quelques dizaines de milliers de lignes : travailler sur un échantillon.
2. Sensible à ν, γ et à l'échelle ; peu explicable.

5. Autoencodeur

Principe : un réseau de neurones apprend à recopier les transactions normales en passant par un goulot étroit. Il doit donc en retenir l'essentiel.



Entraîné uniquement sur des transactions normales. Une transaction normale est bien reconstruite : erreur faible.
Une transaction atypique ne ressemble à rien de ce qu'il a appris : erreur de reconstruction élevée $\Rightarrow$ alerte.

```
from sklearn.neural_network import MLPRegressor
from sklearn.preprocessing import StandardScaler
sc = StandardScaler().fit(X_normales)
ae = MLPRegressor(hidden_layer_sizes=(8, 3, 8), max_iter=500)
ae.fit(sc.transform(X_normales), sc.transform(X_normales))  # entrée = cible
Xt = sc.transform(X_test)
erreur = ((Xt - ae.predict(Xt)) ** 2).mean(axis=1)  # score d'anomalie
```

Points forts : gère beaucoup de variables et des relations non linéaires.
Points faibles : demande beaucoup de données ; réglage de l'architecture ; peu explicable.

Précision, rappel, F_β

$$\text{Précision} = \frac{\text{VP}}{\text{VP} + \text{FP}}, \qquad \text{Rappel} = \frac{\text{VP}}{\text{VP} + \text{FN}},$$

$$F_\beta = (1 + \beta^2)\,\frac{\text{Précision}\,\text{Rappel}}{\beta^2\text{Précision} + \text{Rappel}}$$

Pour $\beta > 1$, le rappel est davantage favorisé ; F_2 est courant en fraude, où manquer une fraude coûte plus qu'une fausse alerte.

Coefficient de corrélation de Matthews (MCC)

C'est la corrélation entre classes réelles et prédites. Il utilise les quatre cases de la matrice de confusion et reste fiable avec des classes très déséquilibrées :

$$\mathrm{MCC} = \frac{\mathrm{VP} \cdot \mathrm{VN} - \mathrm{FP} \cdot \mathrm{FN}}{\sqrt{(\mathrm{VP} + \mathrm{FP})(\mathrm{VP} + \mathrm{FN})(\mathrm{VN} + \mathrm{FP})(\mathrm{VN} + \mathrm{FN})}}$$

Il vaut 1 pour un modèle parfait, 0 pour un modèle aléatoire et -1 pour un modèle toujours faux.

AUC ROC et statistique de Mann-Whitney

L'AUC est la probabilité qu'une fraude ait un score plus élevé qu'une transaction normale. Elle se calcule à partir des rangs des scores :

$$\text{AUC} = \frac{\sum_{i:y_i=1} \text{rang}(s_i) - \frac{n_1(n_1+1)}{2}}{n_1 n_0}$$

où n_1 et n_0 sont les nombres de fraudes et de transactions normales.
Précision@k et rappel@k : Si l'équipe peut examiner k alertes par jour : parmi les k transactions au score le plus élevé, combien sont des fraudes (précision@k), et quelle part des fraudes cela représente-t-il (rappel@k) ?

Coût de la fraude

Toutes les fraudes ne se valent pas : manquer une fraude de 2 000 DT est plus grave que manquer une fraude de 5 DT. Avec un coût fixe C_a d'analyse par alerte, le coût associé au seuil τ est :

$$C(\tau) = \sum_{\substack{i : y_i = 1 \\ s_i < \tau}} M_i + C_a \, N_{\text{alertes}}(\tau)$$

où M_i est le montant de la fraude manquée, s_i le score de la transaction et $N_{\text{alertes}}(\tau)$ le nombre total d'alertes.
Le seuil τ se choisit en minimisant ce coût sur un jeu de validation, puis on le confirme sur le test. On compare toujours au coût « sans modèle » (aucune alerte, toutes les fraudes passent).

Récapitulatif : les méthodes de détection d'anomalies

Critère	Isolation Forest	LOF	DBSCAN	One-Class SVM	Autoencodeur
Question posée	Est-il facile à isoler ?	Est-il plus isolé que ses voisins ?	Appartient-il à un groupe dense ?	Est-il dans la frontière du normal ?	Est-il bien reconstruit ?
Résultat	Score entre 0 et 1	Score autour de 1	Groupe ou bruit (-1)	Distance à la frontière	Erreur de reconstruction
Réglages clés	Nombre d'arbres, taille d'échantillon	Nombre de voisins k	ε, MinPts	ν, γ	Architecture du réseau
Volume adapté	Très gros	Moyen	Moyen	Petit à moyen	Très gros
Nouvelles transactions	Oui	Oui (`novelty=True`)	Non	Oui	Oui
Point fort	Rapide, simple, peu de réglages	Détecte les anomalies locales	Groupes de toutes formes	Frontières complexes	Beaucoup de variables, relations non linéaires
Limite	Coupures parallèles aux axes	Lent, sensible à k	Un seul ε pour tous les groupes	Lent, réglage délicat	Données et réglages exigeants
Classe Python	`IsolationForest`	`LocalOutlierFactor`	`DBSCAN`	`OneClassSVM`	`MLPRegressor`, Keras

Vue d'ensemble : quatre familles d'approches

Méthode / exemples	**Labels**	**Détecte**	**Atout**	**Limite**
Règles métier (ex. 3 retraits étrangers en 1 h)	Non	Scénarios de fraude connus	Immédiates, lisibles, justifiables	Rigides ; contournables ; nombreuses fausses alertes
Supervisé (logistique, gradient boosting)	Oui	Fraudes déjà vues	Très performant ; explicable (SHAP)	Étiquettes fiables requises ; ignore les fraudes nouvelles
Non supervisé (IF, LOF, DBSCAN, One-Class SVM, autoencodeur)	Non	Comportements inhabituels	Repère des anomalies nouvelles	Inhabituel $\neq$ frauduleux : nombreux faux positifs

Quel algorithme choisir ?

Votre situation	**Point de départ conseillé**
Des fraudes étiquetées en nombre suffisant	Gradient boosting + seuil fixé selon le coût métier
Peu ou pas d'étiquettes, gros volume	Isolation Forest
Des groupes de clients aux comportements très différents	LOF
Explorer les données et découvrir des groupes	DBSCAN
Volume moyen, frontière du « normal » complexe	One-Class SVM
Très nombreuses variables, gros volume	Autoencodeur
Fraude organisée, blanchiment, comptes liés	Approche par graphes